

### Homework 3: Medical Image Segmentation

#### Step1

1) How many data samples are included in the dataset?

There are a total of 100 data samples. The dataset consists of 80 training samples and 20 testing samples.

2) Which problem will this dataset try to address?

This dataset solves the problem of retinal blood vessel segmentation which is a high-order task in the domain of medical image analysis. Accurate retinal blood vessel is a key problem in most retinopathy conditions such as diabetic retinopathy, glaucoma, and age-related macular degeneration. This dataset may help researchers or doctors to diagnose, screen, and monitor these diseases in order to treat and predict the outcome more accurately which ultimately benefits the patients.

3) What is the dimension ranging in the dataset?

The images contained in the data set are of variable dimension, reflecting the real-world diversity of retinal imaging. Retinal fundus images are often of different sizes due to the modalities of imaging devices and the resolution settings at which images are captured. This diversity calls for uniform pre-processing whereby every image will be uniformly resized to a specified dimension (for example, I have taken 512 x 512) rendering it suitable for feeding into the deep learning model. Thus several orchestrated pre-processing methods have been proposed, increasing each image to the required dimension-free input process.

4) Does the dataset have any missing information? E.g., missing features.

The dataset does not have missing values; all images and labels/masks are present which makes it suitable for supervised learning tasks.

5) What is the label of this dataset?

The labels in this dataset are level segmentation masks on each retinal fundus image. The said masks are binary, with each pixel classified as a blood vessel: 1 or background class: 0. The binary segmentation masks permit high pixel-wise classification precision that enables the model to accurately distinguish between blood vessel structures across a variety of retinal images.

6) How many percent of data will you use for training, validation and testing?

A split of 70% to train, 15% to validate, and 15% to test I have made. Such an allocation did allow enough training data for the model to learn well, aside from a validation set that is reserved for tuning hyperparameters and a separate test set to evaluate the final model's performance. Such a split assured a well-generalized model not being overfitted on specific images or vessel patterns.

7) What kind of data pre-processing will you use for your training dataset?

For preprocessing the data for my models, I have performed the following steps:

1. Resizing: Adjust the images and masks to a uniform size (e.g., 256x256 pixels) to ensure consistent input dimensions for the model.
2. Normalization: Scale pixel values to the range [0, 1] by dividing by 255.0, which helps improve model convergence during training.
3. Data Augmentation: I will try to apply techniques such as rotation, flipping, and zooming to increase the diversity of the training data and enhance the model's robustness to variations.
4. Binary Mask Processing: Ensure that the masks are in binary format, with blood vessel pixels marked as 1 and background pixels as 0, if necessary.

### Step 2: Model

| Model          | IoU    | Dice   |
|----------------|--------|--------|
| U-Net 2 layers | 0.7369 | 0.5834 |
| U-Net 3 layers | 0.6972 | 0.5352 |
| Any Other?     | -      | -      |

### Step 3: Objective

Which loss function are you going to use to train your model?

**Answer :** My model is being trained based on the binary cross-entropy loss function; it is really appropriate for binary segmentation tasks such as Retina Blood Vessel Segmentation. It computes the difference between predicted and real binary labels, where 1 is a blood vessel and 0 is the background. Therefore, this is an ideal function to conduct pixel-wise classification in medical image processing. While precise segmentation can diagnose the retinal disease, binary cross-entropy allows the model to focus on learning the fine details of vessel boundaries by minimizing classification errors at each pixel.

### Step 4: Optimization

Reason for using the optimizer:

I used **the Adam optimizer** for training the U-Net model because it adapts the learning rates for each parameter, thereby helping the model converge faster and more efficiently. This includes most of the benefits of AdaGrad and RMSProp by keeping track of both the means and variances of the gradients. This allows it to handle sparse gradients and high-dimensional data very nicely. This is particularly useful in Retina Blood Vessel Segmentation, where fine details and boundaries need to be captured. Besides, very minimum hyperparameter tuning is required by the Adam algorithm. Hence, for this kind of medical image segmentation tasks, where precision and stability are of the utmost importance, the method is pretty robust and convenient.

### Step 5: Model selection

Have you experienced different learning rates?

I experimented with various learning rates for training the U-Net model on the Retina Blood Vessel Segmentation dataset. With a high learning rate, convergence is very fast, and the model gets stuck in suboptimal minima. Too low of a learning rate made convergence very slow and led to poor learning of the model. A moderate learning rate, along with a scheduler like ReduceLROnPlateau, worked best ultimately. This approach reduces the learning rate gradually when the validation loss reaches its plateau, therefore enabling finer tunings in the later stages of training, which in turn yields better segmentation performance.

| Model          | Dice and IoU with normalization       | Dice and IoU without normalization    |
|----------------|---------------------------------------|---------------------------------------|
| U-Net 2 layers | Dice Score: 0.7369, IoU Score: 0.5834 | Dice Score: 0.7219, IoU Score: 0.5648 |
| U-Net 3 layers | Dice Score: 0.6972, IoU Score: 0.5352 | Dice Score: 0.7104, IoU Score: 0.5509 |
| Any Other?     | -                                     | -                                     |

How do you avoid overfit your model and underfit your model?

#### Answer:

To avoid overfitting and underfitting in the U-Net model for Retina Blood Vessel Segmentation, I am implementing several strategies to achieve the right balance:

**Data augmentation:** These have to do with applying various transformations on the training images, like rotations, flips, and intensity shifts. That helps to artificially increase the dataset. This has the effect of smoothing the model and allowing it to generalize much better to unseen data, thus reducing overfitting.

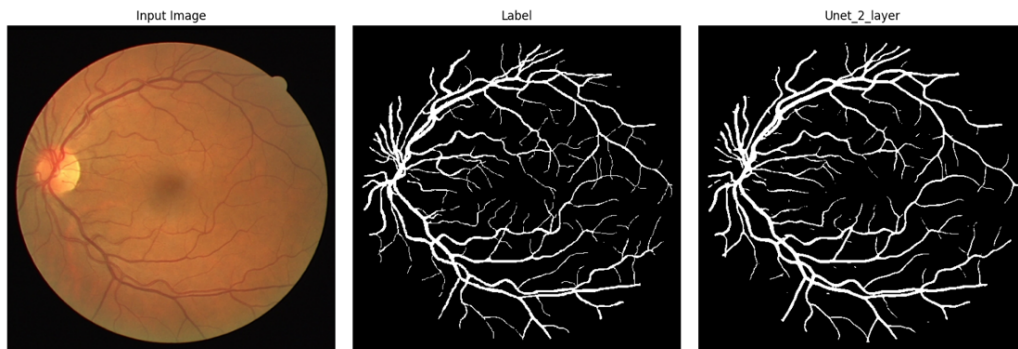
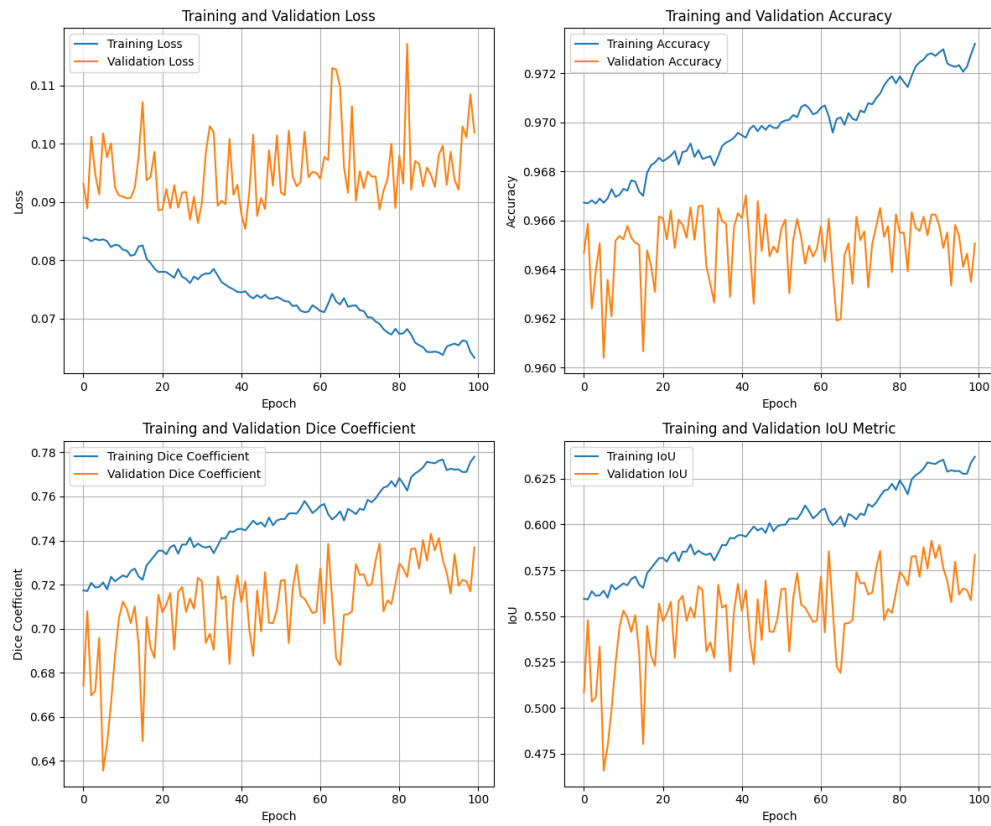
**Learning Rate Scheduling:** By using a learning rate scheduler (e.g., ReduceLROnPlateau), the learning rate is reduced when validation loss plateaus, allowing the model to converge to an optimal solution without overfitting due to a high learning rate.

**Sufficient Training Data:** It avoids underfitting by having enough training data such that the model learns complex patterns with sufficient examples. Data augmentation helps simulate this, too.

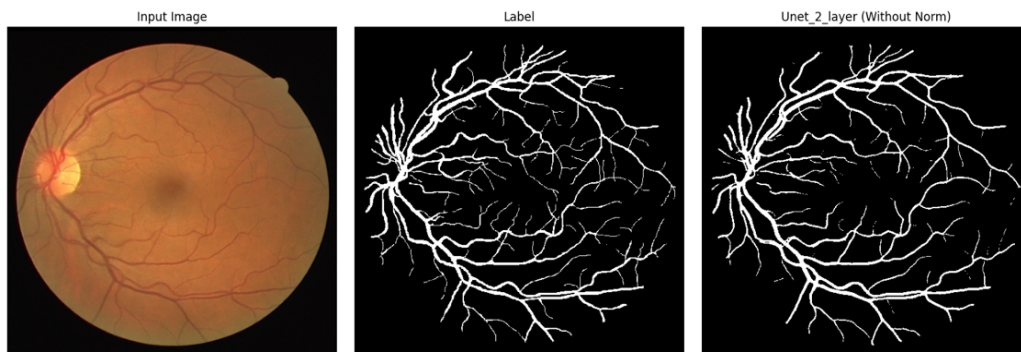
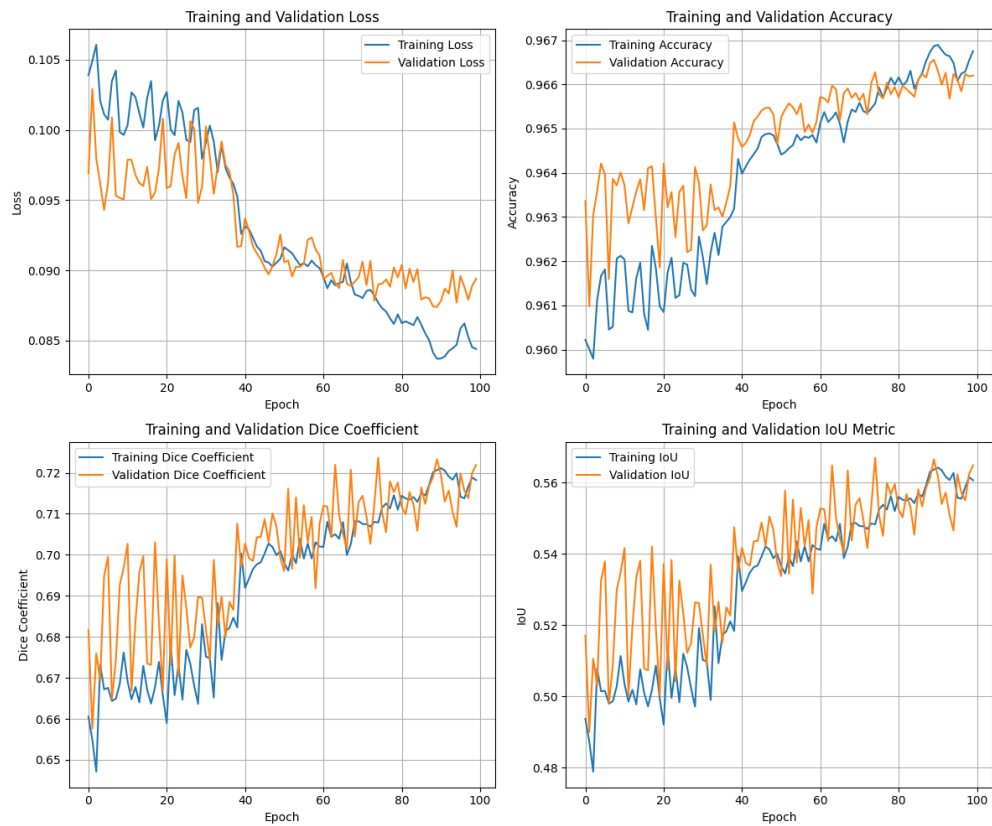
By combining these techniques, I aim to strike a balance where the model is neither overfitting (memorizing the training data) nor underfitting (failing to capture the underlying patterns in the data), resulting in a more robust and accurate segmentation model.

## Step 6: Model demo

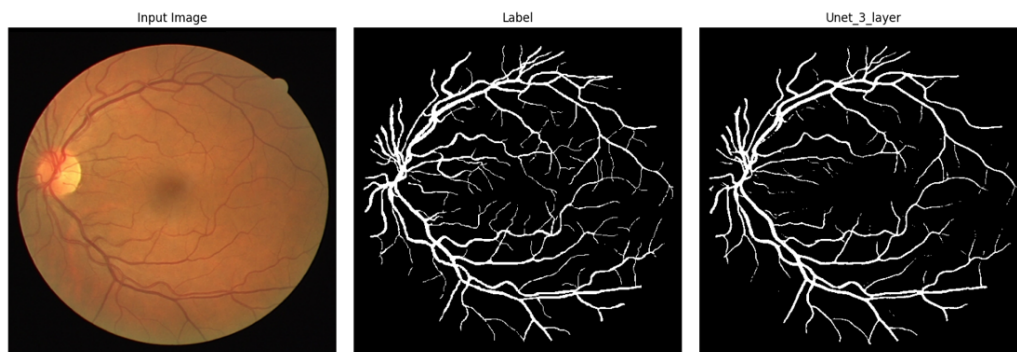
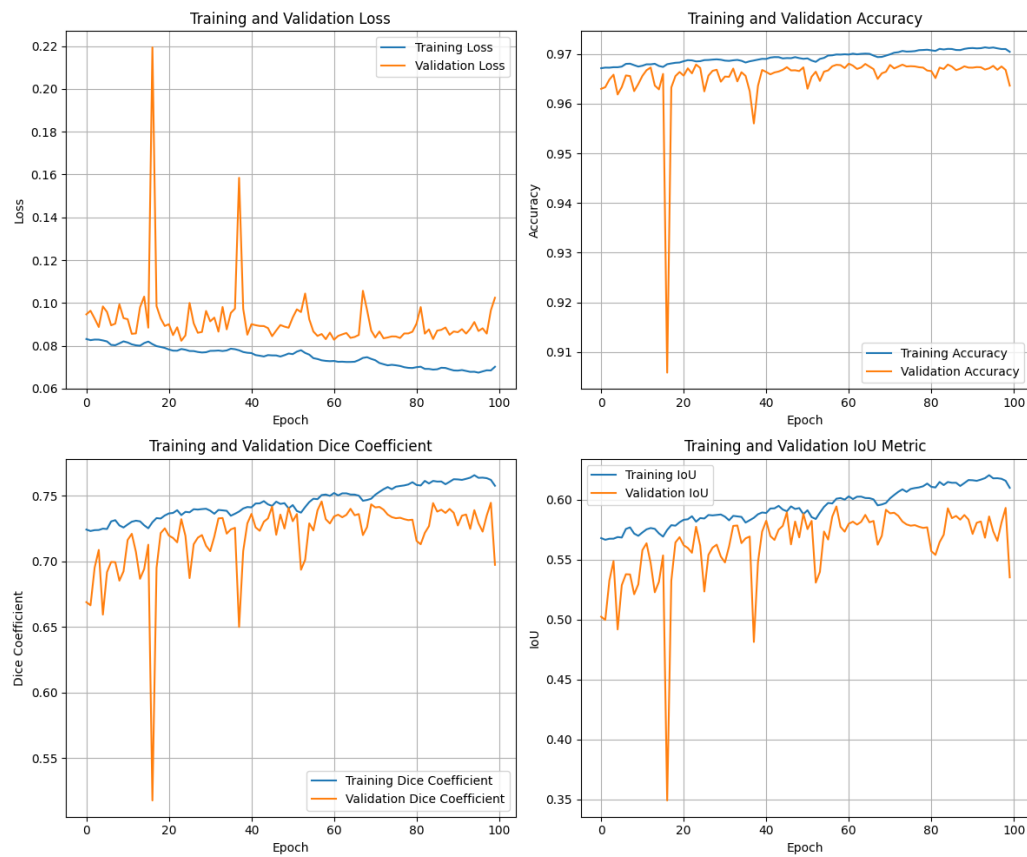
- **U-Net (2 layers) (With Norm) ( Best Model)**



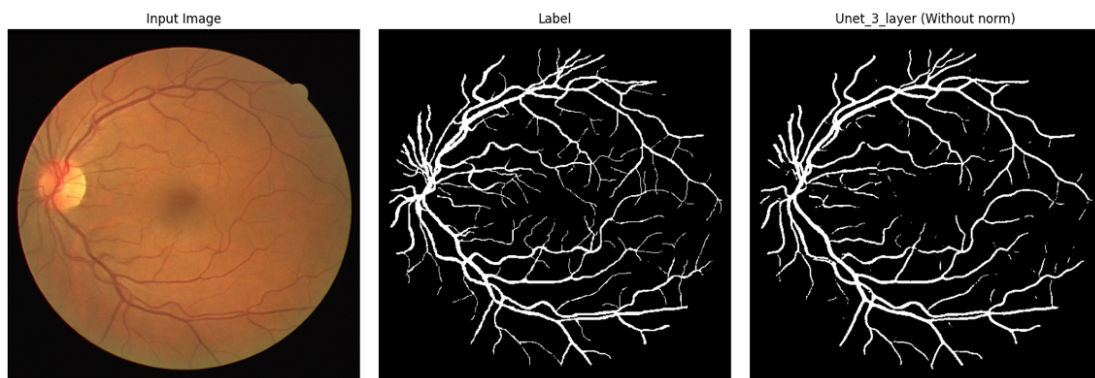
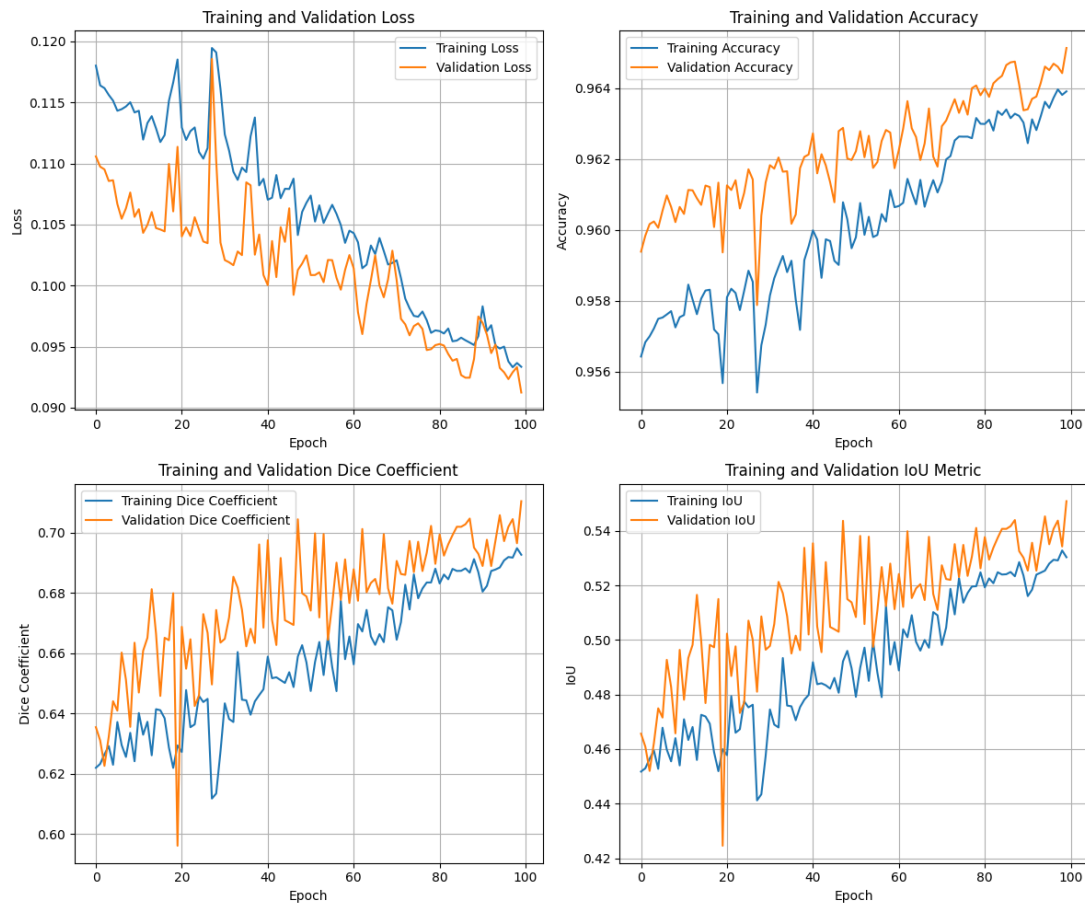
- **U-Net (2 layers) (Without Norm)**



- **U-Net (3 layers) (With Norm)**



- **U-Net (3 layers) (Without Norm)**



## Conclusion

